21 · Learning Roadmap and Interview Study Plan

> **Goal:** convert the repository into an executable plan: know what each level expects, measure your gaps with evidence, choose a time-boxed study plan that includes the new chapters, read the right primary sources, and practise the interview loop deliberately.

Table of Contents

- 21.1 Skill Ladder and Self-Assessment Rubric
- 21.2 Study Plans: 72 Hours, 2 Weeks, 8 Weeks
- 21.3 Reading List: Foundational Papers and Books
- 21.4 The Interview Loop: Rounds, Rubrics, Practice and Offer-Level Signals

The month-long schedule already lives in `COURSE-30-DAY.md` and the README roadmaps; this chapter does not repeat them. It adds time-boxed variants that include chapters 13-22, a rubric for choosing what to study, and a loop-practice protocol. Questions to practise with are in chapter 22 and the projects to build are in chapter 20.

21.1 Skill Ladder and Self-Assessment Rubric

Conceptual Deep Dive

Levels differ in kind, not only in breadth. A **junior** engineer completes a defined task with review. A **mid-level** engineer owns a component, explains mechanisms and follows established patterns. A **senior** engineer owns a system end to end, chooses between options with measured trade-offs, anticipates failure modes and operates what they ship. A **staff** engineer frames ambiguous problems across teams, sets standards and platform direction, quantifies business impact and raises the level of others. In MLOps interviews the expectation shows up as scope (task, component, system, organisation), autonomy (spec given, spec proposed, what not to build), and evidence (definitions, mechanisms, numbers, second-order effects).

Score yourself across ten domains: data, experiments, registry, CI/CD/CT, serving, deployment, monitoring, infrastructure, security and governance, and platform and leadership. Use 0-4 anchors that need observable evidence: 0 unfamiliar; 1 can explain the term; 2 can build it with a guide; 3 can build and debug it unaided and can name its failure modes; 4 can teach it and has made trade-off decisions on it in a real or project setting. A score above 2 without a written piece of evidence (a project, a drill, an incident) counts as 2.

Domain	Study first	Then
Data	2 (legacy), 2.15-2.17	13.2, 14.2
Experiments	3 (legacy), 3.15-3.16	15.4
Registry	4 (legacy), 4.15-4.16	10.16, 17.4
CI/CD/CT	5 (legacy), 5.15-5.17	13.1, 13.6
Serving	6 (legacy), 6.16-6.18	14.5, 19.3
Deployment	7 (legacy), 7.17-7.18	20.3
Monitoring	8 (legacy), 8.16-8.18	11.17, 11.18
Infrastructure	9 (legacy), 9.16-9.18	15.1-15.5

Domain	Study first	Then
Security and governance	10 (legacy), 10.15-10.16	17.1-17.5
Platform and leadership	1.17, 16.1-16.5	12.17

The gap for a domain is (target level score minus your score) times the interview weight for your role. Platform roles weight infrastructure and platform higher; applied ML roles weight data, monitoring and experimentation. Common level mismatches: describing tools when the level calls for decisions, answering at component scope in a staff loop, and overrating from tool familiarity (having used MLflow is not the same as designing a registry policy). The opposite error is skipping weak areas because they are uncomfortable. Re-assess every two weeks with fresh evidence, not a mood.

Cross-Connection Scenarios

- The scoring method in 21.1 feeds the plans in 21.2 and the mock scorecard in 21.4.
- Evidence for scores 3-4 comes from the projects in 20.1-20.5.
- Level-appropriate answer depth is illustrated by the stories in 12.17 and the bank in 22.1.
- The domain table above doubles as an index of the whole repository.

Production Anti-Patterns

- Self-rating without evidence: every domain reads as a 3 until a live interview exposes the gap.
- Studying only favourite topics: comfortable hours accumulate while the highest-weight gap stays open.
- No re-assessment: the plan is never corrected by what the mocks revealed.

Minimal Working Code

A self-assessment calculator with level estimate and weighted gap list.

```
WEIGHTS = {"data": 3, "experiments": 2, "regi
stry": 2, "cicd_ct": 3, "serving": 3,
          "deployment": 3, "monitoring": 3, "infra": 2, "security_gov
": 2, "platform": 2}
CHAPTERS = {"data": "2, 13.2", "experiments": "3, 15.4", "re
gistry": "4, 10.16",
           "cicd_ct": "5, 13", "serving": "6, 14", "deploym
ent": "7",
           "monitoring": "8, 11.18", "infra": "9, 15", "sec
urity_gov": "10, 17",
           "platform": "16, 12.17"}
LEVELS = [("staff", 3.4, 2), ("senior", 2.8, 2), ("mid", 2.0, 1), ("ju
nior", 0.0, 0)]

def level_estimate(scores):
    avg = sum(scores[d] * WEIGHTS[d] for d in WEIGHTS) / sum(WEIGHTS.values())
    for name, min_avg, min_dom in LEVELS:
        if avg >= min_avg and min(scores.values()) >= min_dom:
            return name, round(avg, 2)

def gaps(scores, target=3):
    g = [(d, (target - s) * WEIGHTS[d], CHAPTERS[d]) for d, s in scores.items() if s < target]
    return sorted(g, key=lambda x: (-x[1], x[0]))

if __name__ == "__main__":
    me = {d: 3 for d in WEIGHTS}
    me["monitoring"], me["platform"] = 1, 1
    print(level_estimate(me))
    for row in gaps(me):
        print(row)
    assert level_estimate(me) == ("mid", 2.6) and gaps(me)[0][0] == "monitoring"
```

A study-plan generator that allocates hours in proportion to gap times weight.

```
def allocate(gap_list, total_hours):
    total = sum(g for _, g, *_ in gap_list)
    return {d: round(total_hours * g / total, 1) for d, g, *_ in gap_list}

if __name__ == "__main__":
    plan = allocate([("monitoring", 6, "8"), ("platform", 4, "16"
), ("data", 2, "2")], 24)
    print(plan)
    assert plan == {"monitoring": 12.0, "platform": 8.0, "data": 4.0}
    assert abs(sum(plan.values()) - 24) < 0.2
```

Interview Angle

Q: How do expectations differ between senior and staff MLOps interviews? **A:** Senior answers pick between options using measured trade-offs and cover operations: monitoring, rollback and cost for a system they own. Staff answers add scope and constraint: what not to build, standards across teams, migration and deprecation, and the organisational cost of each option. Trade-off: staff answers risk vagueness without numbers. Check: does the answer contain arithmetic, a named alternative, and a measurable success criterion?

Q: How would you assess your own readiness honestly? **A:** Score ten domains with written evidence, run timed mocks scored by someone else or against a checklist, and compare. Trade-off: more mocks cost time but expose gaps early. Measurable check: the average mock score per dimension trending up, no domain below 2, and a second person reproducing at least one of your project results.

21.2 Study Plans: 72 Hours, 2 Weeks, 8 Weeks

Conceptual Deep Dive

Choose a plan by calendar and starting point, then adjust with the gaps from 21.1. COURSE-30-DAY.md remains the default month-long path; the three variants below fit shorter windows and include the new chapters. All three share rules: build something each day, do spaced repetition on rapid-fire tables, and run a mock early enough to change the plan.

72 hours. Plan for roughly 30 focused hours inside the window, not 72. Block 1 (about 12 hours): Module 12 sections 1-8 (legacy) for the framework and numbers, then 12.16, and the core of modules 1, 6, 7 and 8. Block 2 (about 10 hours): two full designs spoken aloud with the clock (22.2), then three scenarios from 22.4. Block 3 (about 8 hours): your two weakest domains from 21.1, the story bank in 12.17 and a final mock. Skip chapters 15, 18.3-18.5, 19 and the reading list beyond a summary. Rapid-fire: Module 12 section 6 (legacy) tables twice.

2 weeks. Week 1: data and experiments, registry and CI/CD/CT, serving, deployment (chapters 2-7 plus 13). Week 2: monitoring, infrastructure, security and governance (8-10, 14, 17), then chapter 22 and mocks. Read in the morning, build one lab from chapter 20 in the evening, and review cards daily. Place checkpoints on days 4, 8 and 12.

8 weeks, for career switchers. Weeks 1-2: foundations and a first pipeline (20.1). Weeks 3-4: features, serving and CI/CD (20.2, 20.3). Weeks 5-6: monitoring, infrastructure and the closed loop (20.4). Week 7: platform, cost and domains (16, 18). Week 8: capstone (20.5), mocks twice, story bank.

Retention. Active recall beats rereading. Convert rapid-fire tables to flashcards, teach each section back in one minute without notes, and whiteboard one design with a timer weekly. Track mock scores and weak-area counts per week; if a weak area has not improved in two checkpoints, change the method (build it, not read it).

Edge cases: a plan that ignores weekday constraints (generate it from real available hours, below) and cramming without practice, which creates recognition without recall.

Cross-Connection Scenarios

- Gap-driven allocation of hours: 21.1; mock cadence and scoring: 21.4 and 22.1.
- Labs by day come from 20.1-20.5; reading assignments from 21.3.
- The design and incident drills named in the 72-hour plan are 22.2 and 22.4.
- Estimation fluency, needed in every plan, is 12.16.

Production Anti-Patterns

- Reading without building: knowledge stays at the recognition level and collapses when asked to design.
- No mock interviews: the first timed performance happens in the real loop.
- Memorising answers instead of trade-offs: a changed constraint breaks the memorised path.

Minimal Working Code

A schedule generator that fills days from available hours and a section catalogue.

```
CATALOGUE = {"monitoring": [("8.16", 2), ("8.
17", 2), ("8.18", 2)],
             "serving": [("6.16", 2), ("6.17", 2), ("6.18", 2)],
             "data": [("2.15", 2), ("2.17", 2)]}

def schedule(hours_per_day, weak, catalogue=CATALOGUE):
    queue = [(s, h) for d in weak for s, h in catalogue.get(d, [])]
    plan = []
    for day, cap in enumerate(hours_per_day, 1):
        items, left = [], cap
        while queue and queue[0][1] <= left:
            s, h = queue.pop(0)
            items.append(s)
            left -= h
        if cap > 0 and not items and queue:
            s, h = queue[0]
            queue[0] = (s, h - cap)
            items.append(s + " (part)")
        plan.append((day, items))
    return plan, [s for s, _ in queue]

if __name__ == "__main__":
    plan, left = schedule([2, 4, 0, 3, 4], ["monitoring", "serving"])
    for day, items in plan:
        print(day, items or "rest")
    assert plan[2][1] == [] and left == [] and plan[4][1] == ["6.17", "6.18"]
```

A Leitner-box scheduler with a simulated 14-day run.

```
import random

INTERVAL = {1: 1, 2: 2, 3: 4}      # days until the next review per box

def simulate(cards, days=14, p_correct=0.7, seed=3):
    rng = random.Random(seed)
    box = {c: 1 for c in cards}
    due = {c: 1 for c in cards}
    reviews = []
    for day in range(1, days + 1):
        n = 0
        for c in cards:
            if due[c] <= day:
                n += 1
                box[c] = min(3, box[c] + 1) if rng.random() < p_correct else 1
                due[c] = day + INTERVAL[box[c]]
        reviews.append(n)
    return box, reviews

if __name__ == "__main__":
    cards = [f"card{i}" for i in range(20)]
    box, reviews = simulate(cards)
    print(reviews)
    assert sum(reviews) < len(cards) * 14 and max(box.values()) <= 3
```

Interview Angle

Q: You have 72 hours. What do you study and what do you skip? **A:** Study the framework and numbers (Module 12, 12.16), two end-to-end designs spoken aloud, serving, deployment and monitoring basics, and your two weakest domains, then one mock. Skip distributed training detail, niche domains and paper reading. Trade-off: breadth of safe coverage against depth in one area. Check: a timed design that states requirements, numbers, failure modes and rollback within 45 minutes.

Q: How do you practise system design without a partner? **A:** Draw a random prompt from the bank (21.4), time-box each step, speak aloud and record. Then score against the checklist and write the three weakest points. Trade-off: no live interruption, so add scripted curveballs (22.1). Check: scores per dimension across at least four recordings, plus answers that include arithmetic and a named rejected alternative.

21.3 Reading List: Foundational Papers and Books

Conceptual Deep Dive

This is a curated list; it does not summarise paper contents beyond a generic purpose, and it contains only what is listed. Read for three things: the problem statement, the failure the work prevents, and one sentence you can say in an interview. Links, editions and availability change, so search by title and author and verify the edition you are reading. Papers older than current tooling still matter, because the failure modes persist while tools change. Do not read linearly: pick by weak domain from 21.1.

Systems and debt.

Title	Authors, year	Supports
Hidden Technical Debt in Machine Learning Systems	Sculley et al., 2015	Ch 1, 16
Machine Learning: The High-Interest Credit Card of Technical Debt	Sculley et al., 2014	Ch 1
The ML Test Score	Breck et al., 2017	Ch 13
Software Engineering for Machine Learning: A Case Study	Amershi et al., 2019	Ch 1, 16
Challenges in Deploying Machine Learning: a Survey of Case Studies	Paleyes et al., 2020	Ch 1, 11
Rules of Machine Learning	Zinkevich, Google	Ch 1, 12
TFX: A TensorFlow-Based Production-Scale Machine Learning Platform	Baylor et al., 2017	Ch 5, 16
Data Validation for Machine Learning	Breck et al., 2019	Ch 2, 13.2

Evaluation, drift, experimentation.

Title	Authors, year	Supports
Beyond Accuracy: Behavioral Testing of NLP Models with CheckList	Ribeiro et al., 2020	13.4
Failing Loudly	Rabanser et al., 2019	Ch 8
Detecting and Correcting for Label Shift with Black Box Predictors	Lipton et al., 2018	8.16
On Calibration of Modern Neural Networks	Guo et al., 2017	17.5
Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data	Deng et al., 2013	7.17
Confident Learning	Northcutt et al., 2021	2.16

Training and scale. Random Search for Hyper-Parameter Optimization (Bergstra and Bengio, 2012) and Hyperband (Li et al., 2017) support 15.4. ZeRO (Rajbhandari et al., 2020), Megatron-LM (Shoeybi et al., 2019) and PyTorch Distributed (Li et al., 2020) support 15.1.

Domain systems. Deep Neural Networks for YouTube Recommendations (Covington et al., 2016) and Practical Lessons from Predicting Clicks on Ads at Facebook (He et al., 2014) support 18.1.

Responsible ML. Model Cards for Model Reporting (Mitchell et al., 2019) and Datasheets for Datasets (Gebru et al., 2018) support 17.4. Equality of Opportunity in Supervised Learning (Hardt et al., 2016) and Fairness Through Awareness (Dwork et al., 2012) support 17.1.

Books.

Title	Authors, year	Supports
Designing Machine Learning Systems	Huyen, 2022	Ch 1, 12, 22
Machine Learning Design Patterns	Lakshmanan, Robinson, Munn, 2020	Ch 6, 12
Reliable Machine Learning	Chen et al., 2022	Ch 8, 11
Designing Data-Intensive Applications	Kleppmann, 2017	Ch 2, 14
Site Reliability Engineering	Beyer et al., 2016	8.17, Ch 11
The Site Reliability Workbook	Beyer et al., 2018	8.17
Trustworthy Online Controlled Experiments	Kohavi, Tang, Xu, 2020	7.17
Accelerate	Forsgren, Humble, Kim, 2018	Ch 5, 16
Team Topologies	Skelton and Pais, 2019	16.1
Fundamentals of Data Engineering	Reis and Housley, 2022	Ch 2
Fairness and Machine Learning	Barocas, Hardt, Narayanan	17.1

Edge cases: reading depth against time (read abstract, introduction and conclusion first; go deeper only for your weak domain) and papers you cannot access (use the authors' public pages or talks, and verify the claim before repeating it).

Cross-Connection Scenarios

- Pair papers with the chapter named in the last column; for instance, read the debt paper beside Module 01 section 3 (legacy).
- Use gaps from 21.1 to choose which row to read next; log progress with the tracker below.
- Behavioural testing papers connect to 13.4; experimentation books to 7.17 and 7.18.
- Interview answers that cite a paper should follow the frame in 22.5: claim, trade-off, check.

Production Anti-Patterns

- Collecting papers without notes: the reading list grows while recall stays flat.
- Quoting numbers from memory: a wrong figure damages the credibility of a correct idea.
- Reading everything linearly: low-weight material consumes the hours of high-weight gaps.

Minimal Working Code

A reading-list tracker with filtering and a markdown renderer.

```
READING = [
    {"title": "Hidden Technical Debt in Machine Learning Systems", "authors"
:: "Sculley et al.",
     "year": 2015, "chapter": "1", "status": "read"},
    {"title": "The ML Test Score", "authors": "Breck et al.", &qu
ot;year": 2017,
     "chapter": "13", "status": "unread"},
    {"title": "Failing Loudly", "authors": "Rabanser et al.", &qu
ot;year": 2019,
     "chapter": "8", "status": "unread"},
]
```

```
def unread(items, chapter):
    return [i for i in items if i["status"] != "read" and i["chapter"] == c
hapter]

def table(items):
    rows = ["| Title | Authors, year | Status |", "|---|---|---|"]
    rows += [f"| {i['title']} | {i['authors']}, {i['year']} | {i['status']} |" for i in items]
    return "\n".join(rows)

if __name__ == "__main__":
    print(table(READING))
    assert [i["title"] for i in unread(READING, "13")] == ["The ML Test Score&qu
ot;]
    assert unread(READING, "1") == []
```

A note-card generator that flags empty fields.

```
FIELDS = ["problem", "failure_prevented", "in
terview_sentence"]

def card(title, **notes):
    empty = [f for f in FIELDS if not notes.get(f, "").strip()]
    return {"title": title, **{f: notes.get(f, "") for f in FIELDS}, "incomplete
": empty}

if __name__ == "__main__":
    c = card("The ML Test Score", problem="No shared checklist for production readiness&qu
ot;,
             interview_sentence="I use it as a rubric to find untested areas")
    print(c["incomplete"])
    assert c["incomplete"] == ["failure_prevented"]
```

Interview Angle

Q: Which paper best explains why ML systems accumulate debt, and what does it recommend? **A:** Sculley et al., 2015, which argues that ML adds system-level debt beyond ordinary code debt, through entanglement, data dependencies, glue code, configuration and feedback loops (verify details against the paper before quoting). Recommendations amount to treating data and configuration as first-class, testing them, and paying down debt deliberately. Trade-off: speed now versus maintenance later. Check: dependency, test and incident counts per model over time.

Q: Pick a paper from the list and explain how you would apply its idea to a real platform. **A:** Take the ML Test Score (Breck et al., 2017): use its idea of a graded checklist across data, model, infrastructure and monitoring tests as a release rubric. Score each model, publish the score in the registry and require a minimum for higher risk tiers (4.15). Trade-off: rubric overhead versus catching untested areas. Check: share of models above the bar and incidents traced to a test category scored low.

21.4 The Interview Loop: Rounds, Rubrics, Practice and Offer-Level Signals

Conceptual Deep Dive

A typical MLOps loop has five parts: coding or ML basics; ML system design; an MLOps or infrastructure deep dive; a debugging or incident round; and a behavioural or leadership round. Companies vary, and panels sometimes disagree, so treat the list as a map rather than a promise. Each round scores a handful of dimensions: **problem scoping** (requirements and constraints before boxes), **trade-off reasoning** (named alternatives, costs), **depth** (mechanisms and numbers), **operations** (monitoring, rollback, on-call, cost) and **communication** (structure, time use, handling interruption).

Green flags: clarifies the goal and metric first, states assumptions, quantifies, volunteers failure modes and a rollback path, adapts when a constraint changes. Red flags: drawing before clarifying, naming tools with no reason, ignoring labels and feedback loops, defensiveness. Use R-A-D-I-O-M from Module 12 section 1 (legacy) to time-box a design round, and spend the first five minutes on requirements, scale and success metrics only.

Practice protocol. One mock per week for eight weeks, alternating design, incident and behavioural. Record the audio, score yourself afterwards with the dimensions above, and write one change for the next mock. If possible, trade mocks with a peer and use a short feedback template: one thing done well, one missed question, one number that was missing. Prepare six stories (12.17) and a short list of numbers you can derive on the spot (12.16).

Level calibration. Do not negotiate in the interview; listen for the scope of the questions, since they reveal the target level. If the questions are at component scope for a staff role, steer toward system scope by naming an organisational constraint. Keep leveling conversations generic and factual.

Day-of checklist. Sleep, a one-page numbers sheet, water, a prepared question list for each interviewer (22.6). After a bad round, reset: the next round scores independently, so do not carry the mood. Post-interview, write a retro within 24 hours (questions asked, where you stalled) and feed it into the plan from 21.2.

Edge cases: company-specific tool stacks (state the concept and map it to the tool they use) and panel disagreement (you cannot control it; maximise clear signal in each round).

Cross-Connection Scenarios

- Question banks and scoring anchors: 22.1; behavioural frames: 22.6 and 12.17.
- Study cadence and the weekly mock: 21.2; level evidence: 21.1.
- Design and incident prompts for practice: 22.2, 22.3 and 22.4.
- The capstone walkthrough from 20.5 is a ready-made design-round story.

Production Anti-Patterns

- Optimising for trivia: flag syntax and tool minutiae earn little while trade-off reasoning earns most of the score.
- No retro after mocks: the same mistake repeats because nobody wrote it down.
- Starting to draw before clarifying requirements: the design answers a question nobody asked.

Minimal Working Code

A mock-interview scorecard with averages, trend and the lowest dimension.

```
DIMS = ["scoping", "tradeoffs", "depth",
 "operations", "communication"]

def summarise(mocks):
    avg = {d: sum(m[d] for m in mocks) / len(mocks) for d in DIMS}
    half = len(mocks) // 2
    trend = {d: round(sum(m[d] for m in mocks[half:]) / len(mocks[half:])
                      - sum(m[d] for m in mocks[:half]) / half, 2) for d in DIMS}
    return avg, trend, min(avg, key=avg.get)

if __name__ == "__main__":
    mocks = [
        {"scoping": 2, "tradeoffs": 2, "depth": 3, "operations":
1, "communication": 3},
        {"scoping": 2, "tradeoffs": 3, "depth": 3, "operations":
1, "communication": 3},
        {"scoping": 3, "tradeoffs": 3, "depth": 3, "operations":
2, "communication": 3},
        {"scoping": 3, "tradeoffs": 3, "depth": 4, "operations":
2, "communication": 3},
    ]
    avg, trend, lowest = summarise(mocks)
    print(avg, trend, "practise:", lowest)
    assert lowest == "operations" and trend["scoping"] == 1.0
```

A seeded question-draw tool for solo practice.

```
import random

CATALOGUE = [{"id": "22.2-fraud", "chapter": 22, "level": "L
2"},
```

```
            {"id": "22.2-feed", "chapter": 22, "level": "L2
"},
            {"id": "22.3-platform", "chapter": 22, "level": &quo
t;L3"},
            {"id": "22.4-latency", "chapter": 22, "level": "
;L2"},
            {"id": "22.4-canary-kpi", "chapter": 22, "level": &q
uot;L3"}]

def draw(n, level=None, seed=0):
    pool = [q for q in CATALOGUE if level in (None, q["level"])]
    return [q["id"] for q in random.Random(seed).sample(pool, min(n, len(pool)))]

if __name__ == "__main__":
    print(draw(2, "L2", seed=1))
    assert draw(2, "L2", seed=1) == draw(2, "L2", seed=1)
    assert set(draw(5, "L3")) == {"22.3-platform", "22.4-canary-kpi"}
```

Interview Angle

Q: How do you structure your first 5 minutes of a system-design round? **A:** Restate the problem, ask about the user, the decision being made, scale, latency and freshness needs, label availability and constraints such as privacy, then fix a success metric and say my assumptions aloud. Trade-off: spending time on requirements leaves less for depth, but a wrong problem wastes all of it. Check: by minute five I can state QPS, data volume, a latency target and the metric the business cares about.

Q: What differentiates a hire from a strong hire in MLOps design interviews? **A:** Both produce a working design; the strong hire quantifies it, names rejected alternatives with costs, covers day-2 operations (monitoring, rollback, ownership, cost) unprompted, and adapts when a constraint changes. Trade-off: depth can crowd out breadth, so signpost the plan first. Check: the interviewer's curveball is absorbed with a revised number and a stated consequence rather than a restart.